

Vehicle Damage Detection – Project Documentation

1. Introduction

Vehicle Damage Detection is a deep learning-based computer vision project designed to analyze vehicle images and identify the type and severity of visible damage.

The project provides a Streamlit application where users can upload a vehicle image and receive an AI-generated prediction.

2. Problem Statement

Identifying vehicle damage manually can require time and visual inspection. This project demonstrates how deep learning can be used to assist with the initial identification of visible vehicle damage and its severity.

The system predicts both the damage category and an estimated severity level from an uploaded image.

3. Project Objectives

- Develop an image-based vehicle damage classification system.
- Identify the damage type from predefined classes.
- Estimate the severity level.
- Display prediction confidence.
- Provide probability information for each supported class.
- Build an easy-to-use Streamlit interface.

4. Damage Categories

The application supports the following damage classes:

- Dent
- Scratch
- Broken

5. Severity Categories

The application supports three severity levels:

- Minor
- Moderate
- Severe

6. Technology Stack

Programming Language

Python

Deep Learning Framework

TensorFlow / Keras

Model Architecture

ResNet50 with transfer learning

Web Application

Streamlit

Supporting Libraries

- NumPy
- Pillow
- OS

7. Model Input

The uploaded image is:

1. Converted to RGB.
2. Resized to 224×224 pixels.
3. Converted into a NumPy array.
4. Converted to floating-point values.
5. Normalized by dividing pixel values by 255.
6. Expanded to include the batch dimension.

The resulting image is passed to the trained ResNet50-based model.

8. Prediction Process

The trained model produces two outputs:

Damage Prediction

The model estimates probabilities for:

- Dent
- Scratch
- Broken

Severity Prediction

The model estimates probabilities for:

- Minor
- Moderate
- Severe

The class with the highest probability is selected as the final prediction for each output.

9. Confidence Calculation

The confidence score is calculated from the probability of the selected class.

For example:

Damage Confidence = Highest Damage Probability × 100

Severity Confidence = Highest Severity Probability × 100

The application displays these values as percentages.

10. Confidence Warning System

The application provides three confidence levels:

Low / Moderate Confidence

If either damage or severity confidence is below 70%, the application recommends manual inspection.

Moderate Confidence

If either confidence is between 70% and below 80%, the application recommends manual verification.

High Confidence

If both confidence scores are 80% or higher, the application reports high model confidence.

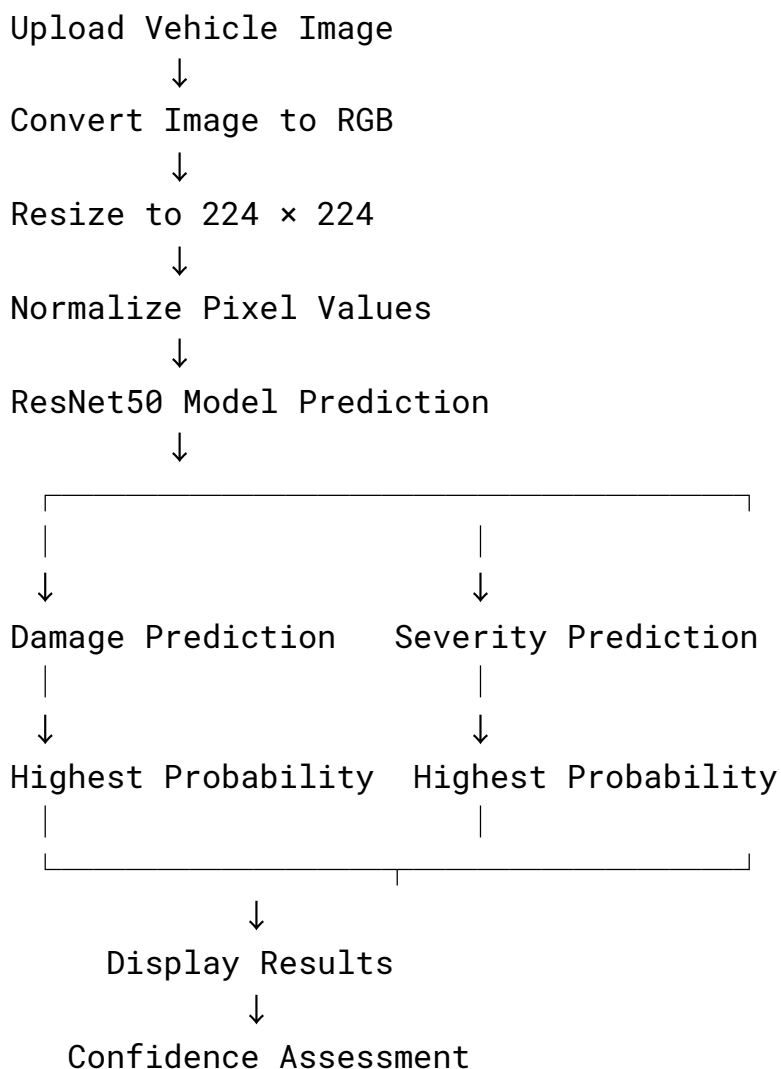
Confidence represents the model's output probability and should not be interpreted as a guarantee of correctness.

11. Streamlit Application Features

The application includes:

- Page configuration with a vehicle-themed title and icon.
- Vehicle image uploader.
- Uploaded image preview.
- Damage detection button.
- Loading indicator while the model performs inference.
- Damage type result.
- Severity result.
- Damage confidence score.
- Severity confidence score.
- Confidence warning.
- Probability details for every damage class.
- Probability details for every severity class.
- Final assessment message.

12. Application Workflow



13. Project Files

Recommended project structure:

Vehicle-Damage-Detection/
|

```
|—— app.py
|—— vehicle_damage_resnet50.keras
|—— requirements.txt
|—— README.md
|—— docs/
    |—— Vehicle_Damage_Detection_Documentation.pdf
```

14. Installation

Install the required Python packages:

```
pip install streamlit tensorflow numpy pillow
```

Alternatively, install from requirements.txt:

```
pip install -r requirements.txt
```

15. Running the Application

Run the following command from the project directory:

```
streamlit run app.py
```

The Streamlit interface will then be available in the browser.

16. Expected Output

After an image is uploaded and analyzed, the application displays:

- Predicted damage type
- Damage confidence
- Predicted severity
- Severity confidence
- Probability for each damage class
- Probability for each severity class
- Final assessment

Example format:

Damage Type: Dent

Severity: Moderate

Damage Confidence: 85.20%

Severity Confidence: 78.40%

The exact values depend on the uploaded image and trained model.

17. Limitations

- The model is limited to the damage classes included during training.
- Predictions depend on image quality and similarity to the training data.
- The system does not replace professional vehicle inspection.
- Confidence scores do not guarantee prediction accuracy.
- Severe variations in lighting, angle, occlusion, or image quality may affect performance.

18. Future Enhancements

Possible future improvements include:

- Adding more vehicle damage categories.
- Improving the training dataset.
- Adding additional severity-related training data.
- Evaluating the model using standard classification metrics.
- Adding image-quality checks.
- Deploying the Streamlit application online.
- Integrating the prediction system into an automated insurance assessment workflow.

19. Conclusion

The Vehicle Damage Detection project demonstrates the use of deep learning and transfer learning for image-based vehicle damage classification.

By combining a ResNet50-based model with a Streamlit interface, the system provides an accessible prototype for predicting damage type and severity from vehicle images. The confidence and probability information helps users understand the model's prediction, while manual verification remains important for practical use.

20. Screenshots

1.

```
image_extensions = (".jpg", ".jpeg", ".png")

for split in ["train", "valid", "test"]:
    split_path = os.path.join(dataset_path, split)

    count = 0

    for root, dirs, files in os.walk(split_path):
        for file in files:
            if file.lower().endswith(image_extensions):
                count += 1

    print(split, ":", count, "images")
```

```
train : 6621 images
valid : 183 images
test : 79 images
```

2.

Classes:

```
1 : minor-dent
2 : minor-dent minor-scratch
3 : minor-dent minor-scratch moderate-dent
4 : minor-dent minor-scratch moderate-scratch
5 : minor-dent moderate-dent
6 : minor-dent moderate-scratch
7 : minor-dent severe-scratch
8 : minor-scratch
9 : minor-scratch moderate-broken
10 : minor-scratch moderate-broken moderate-scratch
11 : minor-scratch moderate-dent
12 : minor-scratch moderate-dent moderate-scratch
13 : minor-scratch moderate-dent moderate-scratch severe-scratch
14 : minor-scratch moderate-scratch
15 : moderate-broken
16 : moderate-broken moderate-dent
17 : moderate-broken severe-broken
18 : moderate-dent
19 : moderate-dent moderate-scratch
20 : moderate-dent severe-broken
21 : moderate-dent severe-broken severe-dent
22 : moderate-scratch
23 : moderate-scratch severe-scratch
24 : severe-broken
25 : severe-dent
```

3.

```
: history_cnn = cnn_model.fit(
    train_ds,
    validation_data=valid_ds,
    epochs=2
)
```

Epoch 1/2

207/207 ————— 203s 978ms/step - accuracy: 0.7934 - loss: 0.9113 - val_accuracy: 0.0055 - val_loss: 6.9538

Epoch 2/2

207/207 ————— 266s 998ms/step - accuracy: 0.8014 - loss: 0.8301 - val_accuracy: 0.0055 - val_loss: 6.7638

4.


```
history_resnet = resnet_model.fit(
    train_ds,
    validation_data=(X_valid, y_valid),
    epochs=2
)
```

Epoch 1/2

207/207 ————— 412s 2s/step - accuracy: 0.7650 - loss: 1.1839 - val_accuracy: 0.6813 - val_loss: 1.4230

Epoch 2/2

207/207 ————— 445s 2s/step - accuracy: 0.7754 - loss: 1.0379 - val_accuracy: 0.6813 - val_loss: 1.4263

5.



Vehicle Damage Detection

Upload a vehicle image to detect the damage type and estimate its severity.

Upload Vehicle Image

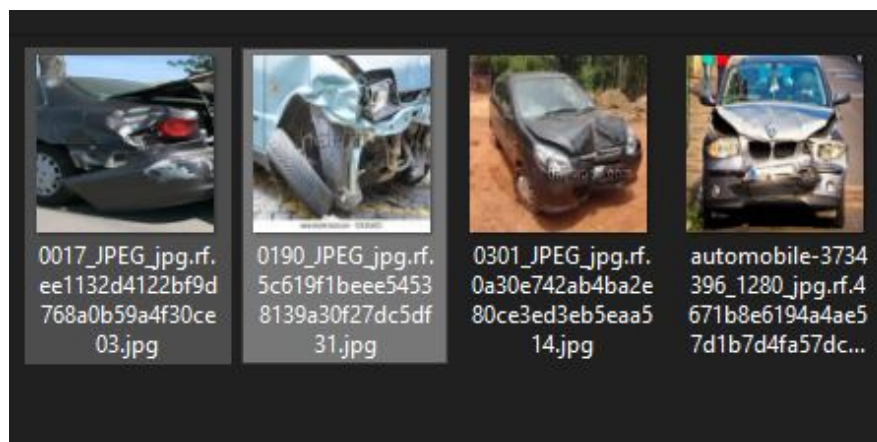


Drag and drop file here

Limit 200MB per file • JPG, JPEG, PNG

Browse files

6.



7.

